Practical Week-12

Task 1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
kiran@k:~/OSSP/ThreadDemo$ cd ~/OSSP
mkdir -p Week12
cd Week12
nano producer_consumer.c
kiran@k:~/OSSP/Week12$ gcc producer_consumer.c -o producer_consumer -pthread

./producer_consumer
Practical Week-12 - Producer Consumer
Counting Semaphores and POSIX Threads
Testing different buffer sizes

Buffer size: 1
Items expected: 100000
Items consumed: 100000
Expected checksum: 5000050000
Actual checksum: 5000050000
Synchronization correctness: PASS
Elapsed time: 10.332533 seconds
Throughput: 9678.17 items/second

Buffer size: 5
Items expected: 100000
Items consumed: 100000
Expected checksum: 5000050000
Actual checksum: 5000050000
Synchronization correctness: PASS
Elapsed time: 2.519929 seconds
Throughput: 39683.66 items/second

Buffer size: 20
Items expected: 100000
Items consumed: 100000
Expected checksum: 5000050000
Actual checksum: 5000050000
Synchronization correctness: PASS
Elapsed time: 0.357484 seconds
Throughput: 279732.86 items/second

Producer-consumer testing completed.
kiran@k:~/OSSP/Week12$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        // Wait for an empty space
        sem_wait(&empty);

        // Enter critical section
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        // Signal that one item is available
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    long sum = 0;

    for (int i = 0; i < ITEMS; i++)
    {
        // Wait for an available item
        sem_wait(&full);
```

```
  GNU nano 8.7.1
        pthread_mutex_lock(&mutex);

        sum += buffer[out];
        out = (out + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        // Signal that one buffer space is available
        sem_post(&empty);
    }

    printf("Consumer processed %d items.\n", ITEMS);
    printf("Consumer sum = %ld\n", sum);

    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    if (argc != 2)
    {
        printf("Usage: %s <buffer_size>\n", argv[0]);
        return 1;
    }

    buffer_size = atoi(argv[1]);

    if (buffer_size <= 0)
    {
        printf("Buffer size must be greater than 0.\n");
        return 1;
    }

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL)
    {
        perror("malloc");
        return 1;
    }

    // Initialize semaphores
    sem_init(&empty, 0, buffer_size);
    sem_init(&full, 0, 0);

    // Initialize mutex
    pthread_mutex_init(&mutex, NULL);

    struct timespec start, end;
```

```
    clock_gettime(CLOCK_MONOTONIC, &start);

    // Create producer and consumer
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    // Wait for threads
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Buffer size = %d\n", buffer_size);
    printf("Execution time = %.6f seconds\n", elapsed);
    printf("Throughput = %.2f items/second\n",
           ITEMS / elapsed);

    // Cleanup
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    free(buffer);

    return 0;
}
```

Task 2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
kiran@k:~/OSSP/Week12$ cd ~/OSSP/Week12
nano deadlock_demo.c
kiran@k:~/OSSP/Week12$ ^C
kiran@k:~/OSSP/Week12$ gcc deadlock_demo.c -o deadlock_demo -pthread -Wall -
Wextra
./deadlock_demo
Practical Week-12 - Deadlock Demonstration

1. Testing deadlock scenario
Thread 1 locks A then requests B.
Thread 2 locks B then requests A.
Deadlock demonstration started.
Thread 1 acquired Resource A.
Thread 2 acquired Resource B.
Thread 2 waiting for Resource A.
Thread 1 waiting for Resource B.
Deadlock detected: threads remained blocked.
Deadlock demonstration terminated safely.

2. Four necessary conditions for deadlock
1. Mutual exclusion: a resource is held exclusively.
2. Hold and wait: a thread holds one resource and waits for another.
3. No preemption: resources cannot be forcibly taken away.
4. Circular wait: threads wait in a circular chain.

3. Prevention using resource ordering
All threads acquire Resource A before Resource B.
Thread 1 acquired Resource A.
Thread 1 acquired Resource B.
Thread 1 released both resources.
Thread 2 acquired Resource A.
Thread 2 acquired Resource B.
Thread 2 released both resources.
Resource-ordering prevention completed successfully.
Deadlock prevention test completed.
kiran@k:~/OSSP/Week12$
```

```
 GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Locked Resource 1\n");

    sleep(1);

    printf("Thread 1: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Locked Resource 2\n");

    sleep(1);

    printf("Thread 2: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Locked Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}
```

```
    return NULL;
}

int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed.\n");

    return 0;
}
```